

SQL Complete Guide

What is sql

In relational databases, we store data in tables that are linked to each other with relations. Each table stores data like customer, order etc. SQL is the language that is used to interact with relational databases.

RDBMS:- SQL, SQL Server, Oracle.

What is NoSQL databases?

They don't understand SQL, they have their own query language. example: MongoDB, Redis etc.

Install MySQL on windows.

[Download link](#)

Parts of each database

1. **Tables:** Where tables are stored.
2. **Views:** Like a virtual table, we can combine data from multiple tables and store it in a view.
3. **Stored Procedures:** We can have these to query data. Procedure to get all books with genre "fantasy"
4. **Functions:** As name explains they can have queries to execute when function is called.

Parts of table

1. **Record:** Rows in table
2. **Column**
3. **Primary key:** Uniquely identify each record in a table. It's a column like customer id etc. Values in the primary keys should be unique.

SHOW:

Used to list all databases.

```
SHOW DATABASES;
```

```
SHOW SCHEMAS;
```

```
SHOW DATABASES LIKE 'test%';
```

 This shows all databases that start with test

For tables

```
USE DATABASE_NAME
```

```
SHOW TABLES
```

```
SHOW TABLES IN DATABASE_NAME
```

```
SHOW TABLES FROM DATABASE_NAME
```

SELECT

Used to select columns from a table

```
SELECT * FROM TABLE_NAME :
```

 Selects all columns from the table

```
SELECT customer_id FROM TABLE_NAME :
```

 Only selects customer_id column

```
SELECT 1,2 :
```

 Two columns 1,2 with values 1, 2

WHERE

Choose which rows

```
SELECT * FROM Customers WHERE customer_id = 1
```

```
-- before a line for comment
```

ORDER BY

Order by a particular column

```
SELECT * FROM customers ORDER BY first_name
```

Descending order

```
SELECT * FROM customers ORDER BY customer_id DESC;
```

Sort by one column then sort the results by another column.

```
SELECT * FROM customers ORDER BY first_name, city ASC;
```

```
SELECT first_name, last_name FROM customers ORDER BY first_name DESC;
```

```
SELECT * FROM order_items WHERE order_id = 2 ORDER BY quantity * unit_price  
DESC;
```

A better example:

```
`SELECT *, quantity * unit_price AS total_price  
FROM order_items WHERE order_id = 2  
ORDER BY total_price DESC;`
```

ARITHMETIC OPERATIONS

```
SELECT last_name, first_name, points, points * 10 + 100 FROM customers;
```

BODMAS is followed.

GIVE NAME TO COLUMNS

```
SELECT last_name, first_name, points, points * 10 + 100 as discount_factor  
FROM customers;
```

for name with spaces

```
SELECT last_name, first_name, points, points * 10 + 100 as "Discount Factor"  
FROM customers;
```

List of distinct col values

```
SELECT DISTINCT state from customers;
```

COMPARISON OPERATORS

```
SELECT * FROM customers WHERE points > 3000;  
SELECT * FROM customers WHERE STATE != 'VA';  
SELECT * FROM customers WHERE STATE <> "VA";  
SELECT * FROM customers WHERE birth_date > '1990-01-01'; date format in sql
```

MULTIPLE CONDITIONS TOGETHER

```
SELECT * FROM customers WHERE birth_date > '1990-01-01' AND POINTS > 1000;  
SELECT * FROM customers WHERE birth_date > '1990-01-01' OR POINTS > 1000;  
SELECT * FROM customers WHERE birth_date > '1990-01-01' OR (POINTS > 1000 AND  
state = 'VA');  
SELECT * FROM customers WHERE NOT (birth_date > '1990-01-01' OR (POINTS >  
1000 AND state = 'VA')); NEGATES  
SELECT * FROM order_items WHERE order_id = 6 AND quantity * unit_price >30;
```

IN OPERATOR

```
SELECT * from customers WHERE state IN ('VA', 'FL', 'GA');
```

We want state to be any of these values

```
SELECT * FROM products WHERE quantity_in_stock IN (48, 38, 72);
```

BETWEEN OPERATOR

Used when comparison involves range of values like what between means.

```
SELECT * FROM customers WHERE points BETWEEN 1000 AND 1800;
```

LIKE OPERATOR

for starting with a particular character:

```
SELECT * FROM customers WHERE last_name LIKE "b%"; (case doesnt matter)
```

any number characters before or after a character: (just contain b)

```
SELECT * FROM customers WHERE last_name LIKE "%b%";
```

ends with a character:

```
SELECT * FROM customers WHERE last_name LIKE "%y";
```

any number of characters replace with ____ followed by y means (underscore_size) plus y size and last should be y:

```
SELECT * FROM customers WHERE last_name LIKE "____y";
```

does not satisfy, here ends with:

```
SELECT * FROM customers WHERE phone NOT LIKE "%9";
```

REGEX OPERATOR

lastname should have field:

```
SELECT * FROM customers WHERE last_name REGEXP "field";
```

here lastname should start with Mac

```
SELECT * FROM customers WHERE last_name REGEXP "^Mac";
```

lastname should end with field

```
SELECT * FROM customers WHERE last_name REGEXP "field$";
```

lastname should have either field or Mac. here pipe should have no space

```
SELECT * FROM customers WHERE last_name REGEXP "field|mac";
```

start with either field mac or rose

```
SELECT * FROM customers WHERE last_name REGEXP "^field|mac|rose";
```

we can also chain

```
SELECT * FROM customers WHERE last_name REGEXP "field$|^mac|^rose";
```

to select that some particular characters from a set should appear before e in lastname

```
SELECT * FROM customers WHERE last_name REGEXP "[aig]e"; matches(ie, ae, ge)
```

same as above but for a sequence of characters

```
SELECT * FROM customers WHERE last_name REGEXP "[a-k]e";
```

NULL OPERATOR

null means the absence of a value

Where customer isn't null:

```
SELECT * FROM customers WHERE phone IS NULL;
```

```
SELECT * FROM customers WHERE phone IS NOT NULL;
```

LIMIT CLAUSE

```
SELECT * from customers ORDER BY first_name LIMIT 3;
```

OFFSET FOR PAGINATION

```
SELECT * from customers ORDER BY first_name LIMIT 4, 3;
```

```
-- PAGE LIMIT 3
```

```
-- PAGE 1: 1-3
```

```
-- PAGE 2: 4-6
```

```
-- PAGE 3: 7-9
```

```
-- OFFSET SKIP_NUM, HOW_MANY
```

INNER JOINS

INNER JOIN returns only the rows with matching values in both tables

Join orders and customers on these column conditions. Orders are shown first:

```
SELECT * FROM orders JOIN customers ON orders.customer_id =  
customers.customer_id;
```

This gives an error. This column is in both the columns. We don't know from which column

to select:

```
SELECT order_date, customer_id FROM orders JOIN customers ON  
orders.customer_id = customers.customer_id;
```

To fix use tablename.column_name;

```
SELECT order_date, customers.customer_id FROM orders JOIN customers ON  
orders.customer_id = customers.customer_id;
```

Using alias to make conditions easier:

```
SELECT order_date, c.customer_id FROM orders o JOIN customers c ON  
o.customer_id = c.customer_id;  
SELECT order_id, oi.product_id, quantity, p.name FROM order_items oi JOIN  
products p ON oi.product_id = p.product_id;
```

Joining Across Databases

```
SELECT * FROM order_items oi JOIN sql_inventory.products p ON oi.product_id =  
p.product_id;
```

SELF JOIN

Joining table with itself.

Here we find managers of the employees.

```
SELECT e.employee_id, e.first_name, e.last_name, m.employee_id AS manager_id,  
m.first_name AS manager_firstname, m.last_name AS manager_lastname FROM  
employees e JOIN employees m ON e.reports_to = m.employee_id;
```

JOIN MULTIPLE TABLES

```
SELECT o.order_id, o.order_date, c.first_name, c.last_name, os.name as status  
FROM orders o JOIN customers c ON o.customer_id = c.customer_id JOIN  
order_statuses os ON o.status = os.order_status_id;
```

COMPOUND JOIN CONDITIONS

Sometimes we cant use a single column to uniquely identify a record. Sometimes we can use combination of values to uniquely identify records.

Composite primary key:More than one column as primary key.

```
SELECT * FROM order_items oi JOIN order_item_notes oin ON oi.order_id =  
oin.order_id AND oi.product_id = oin.product_id;
```

IMPLICIT JOIN SYNTAX

```
-- SELECT * FROM orders -- JOIN -- customers ON orders.customer_id =  
customers.customer_id;
```

Both are similar:

```
SELECT * FROM orders o, customers c WHERE o.customer_id = c.customer_id;
```

OUTER JOINS

1. **Left join** All records from left are returned.

```
SELECT c.customer_id, c.first_name, o.order_id FROM customers c LEFT JOIN  
orders o ON c.customer_id = o.customer_id ORDER BY c.customer_id;
```

2. **Right join** All records from right are returned.

```
SELECT c.customer_id, c.first_name, o.order_id FROM customers c RIGHT  
JOIN orders o ON c.customer_id = o.customer_id ORDER BY c.customer_id;
```

Irrespective of join condition.

The non equal records get null.

We can use outer keyword too

```
SELECT c.customer_id, c.first_name, o.order_id FROM orders o RIGHT OUTER JOIN  
customers c ON c.customer_id = o.customer_id ORDER BY c.customer_id;
```

Outer join with multiple tables

```
SELECT c.customer_id, c.first_name, o.order_id, sh.name as shipper FROM
customers c LEFT JOIN orders o ON c.customer_id = o.customer_id LEFT JOIN
shippers sh ON o.shipper_id = sh.shipper_id ORDER BY c.customer_id;
```

SELF INNER JOIN

```
SELECT e.employee_id, e.first_name, m.first_name as manager FROM employees e
LEFT JOIN employees m ON e.reports_to = m.employee_id;
```

Now this shows even the people who don't have a manager. That means the CEO.

USING CLAUSE

Helpful during joining when the column names are similar. Only works if same column name across multiple tables.

```
SELECT o.order_id, c.first_name FROM orders o JOIN customers c USING
(customer_id);
SELECT o.order_id, c.first_name, sh.name as shipper FROM orders o JOIN
customers c USING (customer_id) LEFT JOIN shippers sh USING (shipper_id);
```

For composite keys?

```
SELECT * from order_items JOIN order_item_notes oin USING (order_id,
product_id);
```

Another example:

```
SELECT p.date, c.name AS client, p.amount, pm.name from payments p JOIN
clients c USING(client_id) JOIN payment_methods pm ON p.payment_method =
pm.payment_method_id;
```

Natural Joins

Database joins based on common values in columns. Very unexpected results can happen so avoid using mostly.

CROSS JOINS

Join every record in the first table with every record in the second table.

```
SELECT c.first_name as customer, p.name as product from customers c CROSS  
JOIN products p ORDER BY c.first_name;
```

Implicit syntax:

```
SELECT c.first_name as customer, p.name as product from customers c, products  
p ORDER BY c.first_name;
```

UNIONS

Using union we can combine records from multiple queries.

```
SELECT order_id, order_date, 'ACTIVE' AS status FROM orders where order_date  
>= '2019-01-30' UNION SELECT order_id, order_date, 'ARCHIVED' AS status FROM  
orders where order_date < '2019-01-30';
```

combining from two tables

```
SELECT first_name FROM customers UNION SELECT name from shippers;
```

COLUMN ATTRIBUTES

1. **INT**: integer values
2. **VARCHAR(max_length)**: variable characters, always take stored length
3. **CHAR(LEN)**: Waste of space if 5 used 45 free.
4. **NN**: not null don't have null values
5. **DATE** date
6. **AI** auto increment: for pk when we add new record increment primary key by one.
7. **default** if left empty what the default value when record is inserted.

INSERT ROW

```
INSERT INTO customers VALUES( DEFAULT, 'John', 'Smith', '1990-01-10', NULL,  
'address', 'city', 'CA', default );  
-- default is for primary key (generates a unique value in this case)  
-- FOR PHONE WE CAN USE NULL OR DEFAULT  
`
```

Another way to add only in the columns we want to (we can list them in any order):

```
INSERT INTO customers ( first_name, last_name, birth_date, address, city,  
state ) VALUES('John2', 'Smith', '1990-01-10', 'address', 'city', 'CA' );
```

INSERTING MULTIPLE ROWS

```
INSERT INTO shippers(name) VALUES('Shipper 1'), ('Shipper 2'), ('Shipper 3');
```

INSERTING HIERARCHIAL ROWS

Across different tables together.

```
INSERT INTO orders (customer_id, order_date, status) VALUES (1, '2019-01-02',  
1);
```

```
INSERT INTO order_items VALUES (LAST_INSERT_ID(), 1, 1, 2.95),  
(LAST_INSERT_ID(), 2, 2, 4.92);
```

-- function is a piece of code we can reuse in sql, here we need for created order id to place in to order_item.

CREATING COPY OF TABLES

```
CREATE TABLE orders_archived AS SELECT * FROM orders;
```

This does not create a primary key. Here the select query is a subquery used inside another query.

Put orders placed before 2019 into another table:

```
INSERT INTO orders_archived SELECT * FROM orders WHERE order_date < '2019-01-
```

```
01';
```

```
CREATE TABLE invoice_archived AS SELECT i.invoice_id, i.number, c.name,  
i.invoice_total, i.payment_total, i.invoice_date, i.due_date, i.payment_date  
FROM invoices i JOIN clients c USING(client_id) WHERE payment_date IS NOT  
NULL;
```

UPDATE DATA IN A SINGLE ROW

```
UPDATE invoices SET payment_total = 10, payment_date = '2019-03-01' WHERE  
invoice_id = 1;
```

```
UPDATE invoices SET payment_total = DEFAULT, payment_date = NULL WHERE  
invoice_id = 1;
```

```
UPDATE invoices SET payment_total = invoice_total * 0.5, payment_date =  
due_date WHERE invoice_id = 3;
```

UPDATING MULTIPLE ROWS

```
UPDATE customers SET points = points + 50 WHERE birth_date < '1990-01-01';
```

USING SUBQUERIES IN UPDATES

```
UPDATE invoices SET payment_total = invoice_total * 0.5, payment_date =  
due_date WHERE client_id = ( SELECT client_id from clients WHERE name =  
"MyWorks" );
```

Update multiple client ids:

```
UPDATE invoices SET payment_total = invoice_total * 0.5, payment_date =  
due_date WHERE client_id IN ( SELECT client_id from clients WHERE state in  
( 'CA', 'NY' ) );
```

DELETE ROWS

```
DELETE FROM invoices WHERE client_id = ( SELECT client_id FROM clients WHERE  
name = 'MyWorks' );
```

VIEW

Queries that has a name and acts as a table;

```
CREATE VIEW full_emps_depts AS SELECT c.eno, c.first_name, last_name, gender  
FROM employees e JOIN current_dept_emp c ON e.emp_no = c.emp_no JOIN  
departments d ON c.dept_no = d.dept_no
```

DISTINCT

get distinct rows from chosen query.

```
SELECT DISTINCT author_id AS id FROM Views WHERE viewer_id = author_id ORDER  
BY id ASC;
```

For multiple columns.

```
SELECT DISTINCT city, country FROM customers;
```

IMPORTANT FUNCTIONS

Length of a string

For mysql: LENGTH()

For SQLServer: Len()

```
SELECT tweet_id FROM tweets WHERE LEN(content) > 15;
```

Max()

Aggregate function to find the max value from a column